Reinforcement Learning

Actor-Critic Algorithms & Value Function Methods

SJTU

Outline

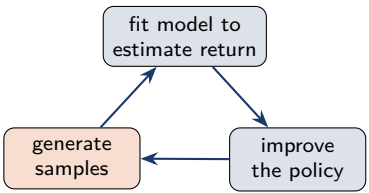
1. Recap: Policy Gradients
2. Value Functions and the Advantage
3. Policy Evaluation
4. The Actor-Critic Algorithm
5. From Actor-Critic to Value-Based Methods
6. Policy Iteration & Dynamic Programming
7. Fitted Q-Iteration
8. Q-Learning
9. Convergence Theory
10. Summary

Recap: Policy Gradient (REINFORCE)

REINFORCE algorithm

1. Sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right) \hat{Q}_{i,t}^\pi$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

$$\hat{Q}_{i,t}^\pi = \sum_{t'=t}^{T} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \quad \text{(reward-to-go)}$$



Problem: $\hat{Q}_{i,t}^\pi$ is a *single-sample* estimate — high variance!

State & State-Action Value Functions

Definitions

- $Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \,\middle|\, \mathbf{s}_t, \mathbf{a}_t\right]$ — total expected reward from $(\mathbf{s}_t, \mathbf{a}_t)$
- $V^\pi(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta}[Q^\pi(\mathbf{s}_t, \mathbf{a}_t)]$ — expected reward from state $\mathbf{s}_t$
- $A^\pi(\mathbf{s}_t, \mathbf{a}_t) = Q^\pi(\mathbf{s}_t, \mathbf{a}_t) - V^\pi(\mathbf{s}_t)$ — how much **better** is $\mathbf{a}_t$ than average?

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \, A^\pi(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

Key insight: the better the estimate of A^π, the lower the variance.

Fitting Only V^π Is Enough

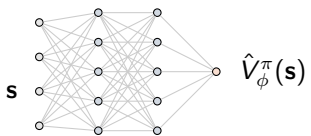
We need $A^\pi = Q^\pi - V^\pi$, but fitting both is expensive. Note:

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \underbrace{\mathbb{E}_{\pi_\theta}\left[\sum_{t'=t+1}^{T} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \,\middle|\, \mathbf{s}_t, \mathbf{a}_t\right]}_{\approx V^\pi(\mathbf{s}_{t+1})}$$

$$\Rightarrow \quad A^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^\pi(\mathbf{s}_{t+1}) - V^\pi(\mathbf{s}_t)$$

Simplification: just fit $V^\pi(\mathbf{s})$ with a neural network (the **critic**).

- Input: state $\mathbf{s}$
- Output: scalar $\hat{V}^\pi_\phi(\mathbf{s})$
- Parameters ϕ (separate from actor θ)



Policy Evaluation: Monte Carlo vs. Bootstrapping

Monte Carlo target

$$y_{i,t} = \sum_{t'=t}^{T} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$$

+ Unbiased
– High variance (long trajectories)

Bootstrapped target

$$y_{i,t} = r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}^{\pi}_{\phi}(\mathbf{s}_{i,t+1})$$

+ Lower variance
– Biased (depends on current $\hat{V}^{\pi}_{\phi}$)

Supervised regression loss (same for both):

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i} \left\| \hat{V}^{\pi}_{\phi}(\mathbf{s}_i) - y_i \right\|^2$$

Historical examples

- **TD-Gammon** (Tesauro 1992): bootstrapped training with self-play
- **AlphaGo** (Silver 2016): value network + Monte Carlo tree search

The Batch Actor-Critic Algorithm

Batch actor-critic algorithm (with discount γ)

1. Sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run policy on environment)
2. Fit $\hat{V}^\pi_\phi(\mathbf{s})$ to sampled reward sums (minimize $\mathcal{L}(\phi)$)
3. Evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}^\pi_\phi(\mathbf{s}'_i) - \hat{V}^\pi_\phi(\mathbf{s}_i)$
4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i)\, \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Online actor-critic

1. Take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$; get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
2. Update $\hat{V}^\pi_\phi$ with target $r + \gamma \hat{V}^\pi_\phi(\mathbf{s}')$
3. Compute $\hat{A}^\pi$; gradient step on θ

- **Batch:** stable, but slow (many samples/update)
- **Online:** fast, but noisy gradients
- **Practical:** parallel workers (A3C)

Discount Factors

For infinite-horizon MDPs ($T = \infty$), values grow unbounded without discounting.

Discounted bootstrapped target:

$$y_{i,t} = r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \gamma \hat{V}^{\pi}_{\phi}(\mathbf{s}_{i,t+1})$$

Discount factor $\gamma \in [0, 1]$ (typically 0.99).

MDP interpretation

γ adds a "death" state: at each step with probability $(1 - \gamma)$, the agent transitions to an absorbing state.

$$\tilde{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = \gamma\, p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$$

Discounted policy gradient

$$\nabla_\theta J \approx \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta\, \hat{A}^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

where $\hat{A}^{\pi} = r + \gamma \hat{V}^{\pi}_{\phi}(\mathbf{s}') - \hat{V}^{\pi}_{\phi}(\mathbf{s})$

Practice: use $\sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$ in the policy gradient (option 1 — later steps still matter).

Architecture Design

Two-network design

- Separate networks for actor π_θ and critic $\hat{V}^\pi_\phi$
- \+ Simple and stable
- – No shared features



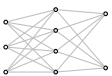
$\hat{V}^\pi_\phi(\mathbf{s})$



$\pi_\theta(\mathbf{a}|\mathbf{s})$

Shared-network design

- One network, two output heads
- \+ Shared feature learning
- – Harder to tune; gradient interference



$\hat{V}^\pi_\phi(\mathbf{s})$

$\pi_\theta(\mathbf{a}|\mathbf{s})$

Can We Remove the Policy Entirely?

The advantage $A^\pi(\mathbf{s}_t, \mathbf{a}_t)$ measures how much better $\mathbf{a}_t$ is than average.

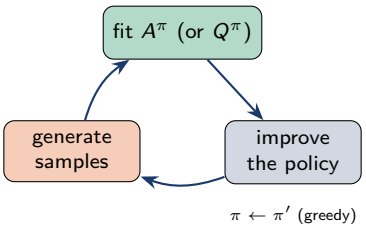
$\Rightarrow \arg\max_{\mathbf{a}_t} A^\pi(\mathbf{s}_t, \mathbf{a}_t)$ is always the **best action**!

Define the **greedy improved policy:**

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \mathbf{a}_t = \arg\max_{\mathbf{a}} A^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

π' is at least as good as π — regardless of $\pi(\mathbf{a}_t|\mathbf{s}_t)$!

Idea: forget the policy gradient. Just do greedy improvement.



Policy Iteration

Policy iteration algorithm:

1. Evaluate $V^\pi(\mathbf{s})$ for all $\mathbf{s}$
2. Set $\pi \leftarrow \pi'$ where $\pi'(\mathbf{a}|\mathbf{s}) = \mathbf{1}[\mathbf{a} = \arg\max_{\mathbf{a}} A^\pi(\mathbf{s}, \mathbf{a})]$

Since $A^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}[V^\pi(\mathbf{s}')] - V^\pi(\mathbf{s})$, we only need V^π!

Bootstrapped policy evaluation (tabular, known transitions):

$$V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma \mathbb{E}_{\mathbf{s}' \sim p}\left[V^\pi(\mathbf{s}')\right]$$

Store $V^\pi(\mathbf{s})$ in a table; run until convergence.



V^π table (16 states)

Value Iteration: Skipping the Policy

Key observation:

$$\arg\max_{\mathbf{a}} A^{\pi}(\mathbf{s}, \mathbf{a}) = \arg\max_{\mathbf{a}} Q^{\pi}(\mathbf{s}, \mathbf{a}) \quad \text{(since } V^{\pi}(\mathbf{s}) \text{ is constant w.r.t. } \mathbf{a})$$

$$Q^{\pi}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}[V^{\pi}(\mathbf{s}')] \quad \text{(simpler!)}$$

Value Iteration Algorithm

1. $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim p}[V(\mathbf{s}')]$
2. $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

Policy (implicit): $\pi(\mathbf{s}) = \arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

\+ No explicit policy; directly computes optimal values

\– Requires known transitions $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ and full state enumeration

Fitted Q-Iteration: Scaling to Large State Spaces

Problem: for large/continuous $\mathcal{S}$, we cannot store $V(\mathbf{s})$ in a table. **Solution:** use a neural network $Q_\phi(\mathbf{s}, \mathbf{a})$.

Fitted Q-Iteration Algorithm

1. Collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy
2. Repeat K times:
   1. $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$ (Bellman target)
   2. $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$

\+ **Off-policy:** works with any collected data
\+ No policy network; no high-variance policy gradient
\– No convergence guarantees with neural nets

Why off-policy? The Bellman target

$$y_i = r_i + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$$

does not depend on which policy generated $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$.

Online Q-Learning

Moving from batch to online: take $N = 1$, $K = 1$, $S = 1$ in fitted Q-iteration.

Online Q-Learning Algorithm

1. Take action $\mathbf{a}_i$ (with exploration), observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

Exploration strategies

ϵ-**greedy:** $\pi(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 - \epsilon & \mathbf{a} = \arg\max Q_\phi \\ \epsilon/(|\mathcal{A}| - 1) & \text{otherwise} \end{cases}$

Boltzmann: $\pi(\mathbf{a}|\mathbf{s}) \propto \exp(Q_\phi(\mathbf{s}, \mathbf{a}))$

Q-Learning with Replay Buffer

1. Store (s, a, s', r) in buffer $\mathcal{R}$
2. Sample mini-batch from $\mathcal{R}$
3. Gradient step on Bellman error

Breaks temporal correlation; improves stability.

Warning: if you code this up naively, it almost certainly won't work! (Next lecture: target networks, etc.)

Does Value Iteration Converge?

Define the **Bellman backup operator**:

$$\mathcal{B}V = \max_{\mathbf{a}} \mathbf{r_a} + \gamma \mathcal{T}_{\mathbf{a}} V$$

where $\mathbf{r_a}$ = reward vector, $\mathcal{T}_{\mathbf{a}}$ = transition matrix.

$\mathcal{B}$ is a contraction (tabular case)

$$\|\mathcal{B}V - \mathcal{B}\bar{V}\|_\infty \leq \gamma \|V - \bar{V}\|_\infty$$

Setting $\bar{V} = V^\star$ (fixed point: $\mathcal{B}V^\star = V^\star$):

$$\|\mathcal{B}V - V^\star\|_\infty \leq \gamma \|V - V^\star\|_\infty$$

Value iteration converges to $V^\star$! Error shrinks by γ each step.

$V^\star$ is unique, always exists, and corresponds to the optimal policy $\pi^\star$.

What Goes Wrong with Neural Networks?

Value iteration: $V \leftarrow \mathcal{B}V$

Fitted value iteration: $V \leftarrow \underbrace{\Pi}_{\text{regression}} \mathcal{B}V$

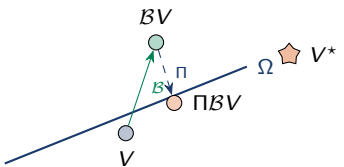
Define projection operator Π:

$$\Pi V = \arg\min_{V' \in \Omega} \frac{1}{2} \sum_{\mathbf{s}} \|V'(\mathbf{s}) - V(\mathbf{s})\|^2$$

Π projects onto the function class Ω (neural nets), contracting in ℓ_2.

Key issue

- $\mathcal{B}$: contraction w.r.t. $\|\cdot\|_\infty$
- Π: contraction w.r.t. $\|\cdot\|_2$
- **$\Pi\mathcal{B}$ is NOT a contraction of any kind**



Fitted iteration can diverge!

(Same issue in actor-critic with bootstrapped targets)

Q-Learning Is Not Gradient Descent

Q-learning update (treating $\mathbf{y}_i$ as constant)

$$\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i) \underbrace{(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)}_{\text{TD error}}$$

where $\mathbf{y}_i = r_i + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$ is treated as a **constant** (no gradient through $\mathbf{y}_i$).

The true gradient of the Bellman error $\mathcal{E}$ would include:

$$\nabla_\phi \mathcal{E} = \underbrace{(Q_\phi - y_i)\nabla_\phi Q_\phi}_{\text{Q-learning keeps}} - \gamma \underbrace{(Q_\phi - y_i)\nabla_\phi Q_\phi(\mathbf{s}', \mathbf{a}'^*)}_{\text{dropped}}$$

- Q-learning is a **semi-gradient** method
- Backpropagating through $\mathbf{y}_i$ causes instability (moving targets)
- In practice: works, but no convergence guarantee

Summary

Actor-Critic Methods

- Critic $\hat{V}_\phi^\pi$ reduces policy gradient variance
- Advantage: $\hat{A}^\pi \approx r + \gamma \hat{V}^\pi(\mathbf{s}') - \hat{V}^\pi(\mathbf{s})$
- Batch vs. online; two-net vs. shared
- Discount γ for stability in infinite horizons

Value-Based Methods

- Policy iteration: evaluate V^π, then greedy improve
- Fitted Q-iteration: neural net, off-policy
- Q-learning: online, ϵ-greedy + replay buffer
- Bellman operator $\mathcal{B}$ is contraction (tabular)
- $\Pi\mathcal{B}$ is NOT a contraction (neural nets)

$$Q_\phi(\mathbf{s}, \mathbf{a}) \leftarrow r + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$$

Next lecture: making Q-learning work in practice

- Target networks
- Double Q-learning
- Prioritized replay
- DQN and variants